

CAF Core — Complete Engineering Guide

Aggregated export — 2026-07-10. Source markdown in repo is unchanged.

Bundle id: `04-caf-engineering-complete-guide`

Included: docs/CAF_CORE_COMPLETE_GUIDE.md

CAF Core — Complete guide (single file)

Purpose: One document that merges the project overview, stack, lifecycles, architecture layers, QC, risk, generation guidance, and repository map. Individual topic files under `docs/` remain the maintained copies for smaller edits; this file is for **printing, onboarding packets, or offline reading**.

Convention: Paths are from the **repository root**. Schema is `caf_core`.

Updated current-state note (2026-07): This merged guide may lag **BVS, New Visual** (`FLOW_VISUAL_FIRST_CAROUSEL` / `execution_mode: new_visual`), and **Why Mimic**. For repo-derived truth, start with `docs/CAF_CURRENT_STATE_CONTEXT_PACK.md`.

Table of contents

1. [What CAF Core is](#)
2. [Tech stack](#)
3. [Domain concepts & entity hierarchy](#)
4. [Lifecycles](#)
5. [End-to-end pipeline \(abbreviated\)](#)
6. [Critical contract: `generation\_payload`](#)
7. [Architecture layers \(detail\)](#)
8. [Quality checks \(QC\)](#)
9. [Risk: policies vs project rules vs brand bans](#)
10. [Generation guidance & learning \(two paths\)](#)
11. [Flow typing & special flow types](#)
12. [HTTP API surface \(index\)](#)
13. [Persistence \(`src/repositories` \)](#)
14. [Repository layout & companion services](#)
15. [Engineering invariants](#)
16. [Split documentation map](#)
17. [External context pack \(ChatGPT / other repos\)](#)

- 18. [Database schema summary](#)
- 19. [Rebuild from documentation](#)
- 20. [Inputs pipeline \(upstream\)](#)

1. What CAF Core is

CAF (Content Automation Framework) is a **content operations / automation platform**. **CAF Core** is the backend: a **Fastify API + PostgreSQL** application that owns **operational truth** for content production—signals, **content jobs**, AI drafts, QC, rendered media, human review, publication placements, and learning data.

Piece	Role
CAF Core (repo root)	API, business logic, <code>caf_core</code> schema
Review app (<code>apps/review</code>)	Next.js UI; not the DB of record
Carousel renderer (<code>services/renderer</code>)	Puppeteer + Handlebars → slide PNGs
Video assembly (<code>services/video-assembly</code>)	ffmpeg → stitched/muxed video
Media gateway (<code>services/media-gateway</code>)	Single port for renderer + assembly

Core is **self-contained**: all planning, job, QC, render, review, publish, and learning state lives in this repo and Postgres; the companion services above only add media rendering and the operator UI.

Problems addressed

- 1. Structured intake (signal packs → candidates with typed fields and lineage).
- 2. Controlled generation (prompts, tenant config, caps, suppression).
- 3. Quality gates (checklists + keyword risk before expensive render).
- 4. Human review (approve / reject / needs edit + rework).
- 5. Media (carousel PNGs, HeyGen/Sora/ffmpeg, Supabase URLs).
- 6. Publishing intent (placements, schedules, outcomes).
- 7. Learning (rules + evidence; planning scores + prompt guidance).

ID conventions (`task_id` , `run_id` , etc.) are defined in `.cursor/rules/caf-domain-model.mdc` .

2. Tech stack

Core API (this repo)

Concern	Choice
Runtime	Node.js ≥ 20

Language	TypeScript (ESM)
HTTP	Fastify 5 — <code>src/server.ts</code>
Validation	Zod (routes + <code>src/config.ts</code>)
DB	PostgreSQL, pg, schema <code>caf_core</code> , migrations/ <code>/*.sql</code>
Tests	Vitest — <code>npm test</code>

LLM & AI

Use	Integration
Text / JSON	OpenAI Chat — <code>OPENAI_API_KEY</code> , <code>OPENAI_MODEL</code>
Structured output	Flow Engine <code>output_schemas</code> ; validation controlled by <code>CAF_OUTPUT_SCHEMA_VALIDATION_MODE</code> (<code>skip</code> / <code>warn</code> / <code>enforce</code> , wins over legacy <code>CAF_SKIP_OUTPUT_SCHEMA_VALIDATION</code>)
Scene clips	OpenAI Videos / Sora when configured — <code>SCENE_ASSEMBLY_CLIP_PROVIDER</code>
Post-approval vision	<code>OPENAI_APPROVAL_REVIEW_MODEL</code>
Top-performer mimic render	<code>MIMIC_IMAGE_PROVIDER</code> (default <code>bfl1</code>); copy uses OpenAI — see section 11

Out-of-process media

Service	Role	Config
Carousel renderer	PNG slides	<code>RENDERER_BASE_URL</code> , <code>CAROUSEL_TEMPLATES_DIR</code>
Video assembly	concat, mux, burn subtitles	<code>VIDEO_ASSEMBLY_BASE_URL</code> , poll timeouts

Core calls these over **HTTP**; it does not run Puppeteer/ffmpeg inside the main API process for those steps.

Optional integrations

Provider	Purpose
HeyGen	Avatar / video agent — <code>HEYGEN_API_KEY</code> , polling <code>HEYGEN_POLL_MAX_MS</code>
BFL / DashScope / NVIDIA NIM	Mimic image edit when <code>MIMIC_IMAGE_ENABLED=1</code> — <code>BFL_API_KEY</code> , <code>DASHSCOPE_API_KEY</code> , <code>NVIDIA_NIM_API_KEY</code>
Supabase	Asset storage — <code>SUPABASE_*</code>
Meta Graph	Publishing when <code>CAF_PUBLISH_EXECUTOR=meta</code>

Review UI

Next.js 14 in `apps/review` — uses `CAF_CORE_URL` and optional `CAF_CORE_TOKEN` .

Deploy hints

Fly (`fly.toml` , `Dockerfile`) — Core API **and** embedded Review admin at <https://caf-core.fly.dev/admin/workbench> (`fly deploy -a caf-core` after `apps/review/` changes). Vercel (`apps/review/vercel.json`) is not the canonical operator UI. Full env list: `ENV_AND_SECRETS_INVENTORY.md` .

3. Domain concepts & entity hierarchy

Concept	Meaning
Project	Tenant (brand); <code>caf_core.projects</code> , keyed by <code>slug</code> in URLs
Signal pack	Research bundle attached to a run; source data for materializing planner rows
Run	One cycle; <code>runs.run_id</code> (text) + UUID id; links <code>signal_pack_id</code>
Content job	Atomic work unit; unique (<code>project_id</code> , <code>task_id</code>); <code>generation_payload</code> is the hub
Draft	One LLM attempt — <code>job_drafts</code>
Asset	Render output — <code>assets</code>
Publication placement	Post intent + outcome — <code>publication_placements</code>

Note (current wiring): run start/planning consumes `runs.planned_jobs_json`

(canonical; dual-written with legacy `candidates_json`) rather than reading

`signal_packs.*_json` directly during `start` . Populate it first via:

- `POST /v1/runs/:project_slug/:run_id/jobs` (materialize from the run's attached signal pack; legacy alias `.../candidates`)

Signal packs store curated rows in `jobs_json` (dual-written with legacy `ideas_json`).

`caf_core.candidates` exists historically; do not assume it is the planning source of truth.

4. Lifecycles

Run (`caf_core.runs`)

Driven by `POST /v1/runs/:project_slug/:run_id/start` → `startRun` in `src/services/run-orchestrator.ts` .

`startRun` requires `signal_pack_id` on the run.

`startRun` also expects `runs.planned_jobs_json` (or legacy `candidates_json`) to already be materialized from the signal pack. Create it first via `POST /v1/runs/:project_slug/:run_id/jobs` while the run is still `CREATED` .

Allowed statuses (SQL check in migration `002`):

`CREATED` , `PLANNING` , `PLANNED` , `GENERATING` , `RENDERING` , `REVIEWING` , `COMPLETED` , `FAILED` , `CANCELLED` .

Phase	Typical status	What happens
Created	CREATED	Attach <code>signal_pack_id</code> before start
Planning	PLANNING	Load pack, build candidates, decideGenerationPlan, insert jobs PLANNED
Generating	PLANNED → GENERATING	After jobs exist
Execution	GENERATING → ...	Job pipeline updates run as configured
Terminal	COMPLETED / FAILED	

Content job (`caf_core.content_jobs`)

Typical path (`job-pipeline.ts`):

PLANNED → GENERATING → GENERATED → (QC) → RENDERING → IN_REVIEW →
APPROVED | REJECTED | NEEDS_EDIT

- After `runQcForJob` , status may be `BLOCKED` , `QC_FAILED` , short-circuit to `REJECTED` / `NEEDS_EDIT` via `routeJobAfterQc` (`validation-router.ts`).
- Default human gate: `CAF_REQUIRE_HUMAN_REVIEW_AFTER_QC` (default true).
- Video may stay `RENDERING` during HeyGen/Sora polls; `RenderNotReadyError` allows retry without marking `FAILED`.

Editorial (`caf_core.editorial_reviews`)

`decision` : `APPROVED` | `NEEDS_EDIT` | `REJECTED` — via `src/routes/v1.ts` .

Publication placements

Statuses: `draft` , `scheduled` , `publishing` , `published` , `failed` , `cancelled` — linked by `(project_id, task_id)` .

Learning rules

`learning_rules.status` : `pending` , `active` , `superseded` , `rejected` , `expired` . `applyLearningRule` sets `active` + `applied_at` .

State log

`caf_core.job_state_transitions` —
`src/repositories/transitions.ts` .

5. End-to-end pipeline (abbreviated)

1. **Ingest** — XLSX upload or CLI → `signal_packs` ; create `run` `CREATED` with `signal_pack_id` .
2. **Materialize candidates** — `POST`
`/v1/runs/:project_slug/:run_id/candidates` writes planner rows into `runs.candidates_json` .
3. **Start run** — `startRun` : `plan` → `upsertContentJob` each selected row.
4. **Process (draft packages)** — run endpoints `POST`
`/v1/runs/:project_slug/:run_id/process` / `/start-and-process`
generate LLM output + QC + diagnostics in background and stop jobs at `GENERATED` (package ready).
5. **Render** — `POST` `/v1/runs/:project_slug/:run_id/render` runs rendering for `GENERATED` jobs in background.
6. **Review** — Human decision; optional rework (`rework-orchestrator.ts`).
7. **Publish** — `publication_placements` ; executor `none` | `dry_run` | `meta` .
8. **Learn** — Rules APIs, metrics, optional editorial cron.

Optional mimic path: when `FLOW_TOP_PERFORMER_MIMIC_*` is enabled and `MIMIC_IMAGE_ENABLED=1` , generate resolves references before copy, render uses image providers + HBS/DocAI overlays — see section 11.

6. Critical contract: `generation_payload`

Stored on `caf_core.content_jobs` . Treat as a **versioned integration surface** for pipeline, Review, and admin.

Contains (non-exhaustive):

- Planner: `signal_pack_id` , `candidate_data` , `prompt_id` , `prompt_version_label` , ...
- LLM: `generated_output`
- QC: `qc_result` (also `qc_status` , `recommended_route` on columns)
- Render: HeyGen/scene/video URLs, `render_state` / nested provider data
- **Mimic (optional)**: `mimic_v1` (render SSOT), `bvs_v1` (Brand Visual System snapshot), `mimic_render_context` , `mimic_job_grounding` , `draft_package_snapshot` with `mimic_carousel_package` for **TP-grounded carousel render flows** (`isTpGroundedCarouselRenderFlow()`)
- Publish helpers: `publish_media_urls_json` , `publish_video_url` , ...

Typed readers (incremental contract hardening)

The `src/domain/` folder carves out typed subsets one slice at a time so new code does not have to repeat unsafe `(x as Record<string, unknown>) ?? {}` casts:

Slice	Module	Helpers
<code>qc_result</code>	<code>generation-payload-qc.ts</code>	<code>qcResultSchema</code> (Zod), <code>mergeGenerationPayloadQc</code> (canonical writer), <code>pickStoredQcResult</code> (tolerant reader)
<code>generated_output</code>	<code>generation-payload-output.ts</code>	<code>pickGeneratedOutput</code> , <code>pickGeneratedOutputOrNull</code> , <code>hasGeneratedOutput</code> — reject arrays/primitives instead of coercing to <code>{}</code>
<code>render_state</code>	<code>content-job-render-state.ts</code>	<code>pickRenderState</code> , <code>hasActiveProviderSession</code> (HeyGen idempotency invariant), <code>isMidProviderPhase</code>
<code>mimic_v1</code>	<code>mimic-payload.ts</code>	<code>pickMimicPayload</code>
<code>bvs_v1</code>	<code>bvs-v1.ts</code>	<code>parseBvsV1</code> , <code>attachBvsToPlannedPayload</code>

Adoption is incremental. Existing call sites keep working; code you touch or write should prefer these helpers. The HeyGen "don't double-submit when `render_state` already holds `video_id` / `session_id` " rule now has a grep-able name: `hasActiveProviderSession` .

7. Architecture layers (detail)

Summary table:

Layer	Responsibility	Primary code
HTTP	Routes, auth	src/server.ts, src/routes/*.ts
Orchestration	Run start → jobs	run-orchestrator.ts
Decision engine	Score, caps, suppression, traces	decision_engine/
Job pipeline	Generate → QC → diagnose → render	job-pipeline.ts
Generation	Prompts, OpenAI, drafts	llm-generator.ts
QC	Checklists + policies + bans	qc-runtime.ts
Rendering	Carousel HTTP, HeyGen, Sora, assembly	heygen-renderer.ts, scene-pipeline.ts, ...
Review / rework	Human + rework	v1.ts, rework-orchestrator.ts
Publishing	Placements	publications.ts
Learning	APIs + compileLearningContexts	learning.ts, learning-context-compiler.ts
Persistence	SQL	repositories/

7.1 HTTP API

- **Entry:** `src/server.ts` — Fastify, CORS, multipart, optional `CAF_CORE_REQUIRE_AUTH` + token; public exceptions: `/health` , `/robots.txt` , renderer template paths.
- **Routes:** `v1` , `runs` , `signal-packs` , `pipeline` , `project-config` , `flow-engine` , `learning` , `publications` , `admin` , `renderer-templates` , `project-integrations` .
- **State:** none durable in-process; all in Postgres.

7.2 Run orchestration

- **startRun** : `CREATED` run + `signal_pack_id` + **materialized** `runs.planned_jobs_json` (canonical; dual-written with legacy `candidates_json`) → delete orphan jobs for run → `PLANNING` → allowed flows (skip `offline-flow-types`) → optional scene router expansion → build candidates

from planner rows → `decideGenerationPlan` → `upsertContentJob` + transitions → `PLANNED` / `GENERATING` /terminal.

- **replanRun** : same file family.

7.3 Decision engine

- **decideGenerationPlan** — input: `GenerationPlanRequest` ; output: selected jobs, dropped list, suppression, `trace_id` → `decision_traces` .
- **Modules:** `scoring.ts` , `ranking_rules.ts` (learning boosts), `kill_switches.ts` , `route_selector.ts` , `prompt_selector.ts` , `default-plan-caps.ts` , `flow-kind.ts` .
- **Planning learning:** only via `getLearningRulesForPlanning` facade (wraps `listActiveAppliedLearningRules`)— `BOOST_RANK` , `SCORE_BOOST` , `SCORE_PENALTY` .

7.4 Job pipeline

- **processRunJobs** , `processContentJobById` , `reprocessJobFromScratch` .
- Stages: offline exit → `GENERATING` → `generateForJob` → `runQcForJob` → `routeJobAfterQc` → diagnostic → render branch → `IN_REVIEW` .
- Failures: `markJobFailedPipeline` ; video polls: `RenderNotReadyError` .

7.5 LLM generation

- **generateForJob** — templates from `flow-engine` , `buildCreationPack` , `getLearningContextForGeneration` (facade), OpenAI, output-schema validation controlled by `schemaValidationMode` (resolved from `CAF_OUTPUT_SCHEMA_VALIDATION_MODE` , legacy fallback `CAF_SKIP_OUTPUT_SCHEMA_VALIDATION`), `job_drafts` insert, merge `generated_output` . In `warn` mode, schema failures are recorded on `generation_payload.schema_validation_warnings` without failing.
- **Special:** scene-bundle flows, `FLOW_IMG_*` blocked in `product-flow-types.ts` , carousel addenda + anti-repetition.

7.6 Rendering

- **Carousel:** HTTP to `RENDERER_BASE_URL` ; `carousel-render-pack.ts` ; retries `CAROUSEL_RENDERER_*` .
- **Video:** `heygen-renderer.ts` , `scene-pipeline.ts` , `sora-scene-clips.ts` ; concat/mux via video-assembly; assets + `render_state` on job.

7.7 Review & rework

- **v1.ts** : queue, detail, editorial decision + overrides (script, slides, HeyGen ids, skip video/image regen, ...).
- **rework-orchestrator.ts** : `executeRework` ; pipeline route triggers.

7.8 Publishing

- **publication_placements** ; routes in `publications.ts` ; `CAF_PUBLISH_EXECUTOR` ; `meta-graph-publish.ts` , `dry-run.ts` ; stable JSON payload exposed at `GET /v1/publications/:project_slug/:id/n8n-payload` (route name is historical — Core does not depend on any specific external executor).

7.9 Learning

- **learning.ts** — rules, evidence, ingest.
- **Cron:** `editorial-analysis-cron.ts` + `EDITORIAL_ANALYSIS_CRON_*` .
- **Global project:** global learning rules are currently **disabled** at the HTTP layer; treat any `caf-global` behavior as not active unless reintroduced by code.
- **Attribution:** `learning_generation_attribution` .
- **Post-approval LLM review:** `src/services/approved-content-llm-review.ts` scores approved jobs and emits **upstream_recommendations** (schema in `src/domain/upstream-recommendations.ts`). Stored on `caf_core.llm_approval_reviews.upstream_recommendations` (migration `025`) and fanned out per item into `learning_observations` with `source_type = "llm_upstream_recommendation"` for a queryable audit trail.
- **Run context snapshot:** `src/services/run-context-snapshot.ts` captures prompt versions, project brand/strategy slices, and learning-guidance fingerprints at end-of-planning. Persisted via `setRunContextSnapshot` into `caf_core.runs.context_snapshot_json` (migration `025`). Snapshot failures never abort a run; they are logged via `pipeline-logger.ts` .
- **Run Logs (Review app):** `/runs` lists every run with prompt/context-snapshot indicators; entry point on the sidebar. Deep-links to `/r/[run_id]` for the per-run review queue.

8. Quality checks (QC)

- **Function:** `runQcForJob(db, jobId, requireHumanReviewAfterQc)` in `src/services/qc-runtime.ts` .
- **Input:** `generation_payload.generated_output` (carousel may be normalized first).

- **Tables:** `flow_definitions` → `qc_checklist_name` / version; rows in `qc_checklists` .
- **Check types:** `required_keys` , `equals` (incl. carousel slide-count helpers), `min_length` , `max_length` , `regex` , `not_empty` ; unknown types default non-blocking pass.
- **Pass:** no blocking failures and no CRITICAL risk finding from policies.
- **qc_result :** `buildQcResultPayload` — score, routes, reasons, blocking lists. Persisted via `mergeGenerationPayloadQc` in `src/domain/generation-payload-qc.ts` — the single write surface. A Zod `qcResultSchema` in the same module validates the shape before touching Postgres; `pickStoredQcResult` is the tolerant reader.
- **Routes:** may yield `BLOCKED` , `DISCARD` , `REWORK_REQUIRED` , `HUMAN_REVIEW` ; `CAF_REQUIRE_HUMAN_REVIEW_AFTER_QC` maps clean passes away from `AUTO_PUBLISH` by default.
- **Separate from:** output-schema validation in `llm-generator.ts` . Rollout is controlled by `CAF_OUTPUT_SCHEMA_VALIDATION_MODE` (`skip` / `warn` / `enforce`); the legacy `CAF_SKIP_OUTPUT_SCHEMA_VALIDATION` flag is still honored as the fallback. In `warn` mode, validation failures are written to `generation_payload.schema_validation_warnings` instead of failing the generation, which lets staging measure impact before flipping to `enforce` .

9. Risk: policies vs project rules vs brand bans

Source	Table / field	Used in <code>runQcForJob</code> ?
Risk policies	<code>caf_core.risk_policies</code>	Yes — <code>listRiskPoliciesForJob(db, flow_type)</code> returns rows where <code>applies_to_flow_type IS NULL OR = flow_type</code> (added in migration <code>024_risk_policies_scope.sql</code>). Keyword scan on <code>JSON.stringify output + riskDetectionTermMatches</code>
Project risk rules	<code>caf_core.risk_rules</code>	No in current <code>qc-runtime</code> — used for profile, CSV import, admin. <code>GET /v1/projects/:slug/risk-qc-status</code> surfaces this asymmetry honestly
Brand bans	<code>brand_constraints.banned_words</code>	Yes — merged into same scan as policies

Planning-time routing uses `route_selector.ts` — different phase from QC `recommended_route` .

Risk honesty surfaces

Every write path that touches **project risk rules** (`caf_core.risk_rules`) now returns (or warns with) the same `risk_qc` notice so operators cannot silently configure unenforced rules:

- GET/POST/DELETE `/v1/projects/:project_slug/project-risk-rules` (preferred) and the deprecated alias `.../risk-rules` , plus POST `/v1/admin/config/risk-rule[/delete]` → attach `risk_qc: riskRulesNotEnforcedNotice()` on the response.
- `importProjectFromCsv` → appends a `warnings[]` entry whenever `applied.risk_rule > 0` .
- GET `/v1/projects/:project_slug/risk-qc-status` → returns a compact payload for operators and admin UI (shape defined in `src/services/risk-qc-status.ts` via `buildRiskQcStatus`):

```
{
  "ok": true,
  "project_slug": "...",
  "qc_uses": ["risk_policies", "brand_banned_words"],
  "project_risk_rules_count": 0,
  "risk_rules_enforced_by_qc": false,
  "has_unenforced_risk_rules": false,
  "message": "...",
  "docs_path": "docs/RISK_RULES.md"
}
```

10. Generation guidance & learning (two paths)

Single facade

- **File:** `src/services/learning-rule-selection.ts` — the **only** place code looks up learning rules today. Two exports map the two mental models:
 - `getLearningRulesForPlanning(db, projectId)` — wraps `listActiveAppliedLearningRules` .
 - `getLearningContextForGeneration(db, projectId, flow, platform, opts)` — wraps `compileLearningContexts` .
- Call sites migrated: `src/decision_engine/index.ts` , `src/services/llm-generator.ts` , `src/routes/learning.ts` (context-preview). A static test in `src/services/learning-rule-selection.test.ts` asserts that none of them bypass the facade.

A. Prompt injection (`getLearningContextForGeneration`)

- **Impl:** `src/services/learning-context-compiler.ts` → `compileLearningContexts` ; reached via the facade.
- **Includes:** active rules where `rule_family === 'generation'` or action matches `GENERATION|GUIDANCE|HINT` ; optional **pending** on editorial rework.
- **Scope:** `scope_flow_type` (wildcards), `scope_platform` .
- **Text:** from `action_payload : guidance , hint , text , message , summary` .
- **Injection:** template placeholders + system prompt appendix ("do not quote verbatim").
- **Caps:** `LLM_LEARNING_*` env vars.

B. Planning-only rules (`getLearningRulesForPlanning`)

- **Impl:** `src/repositories/core.ts` → `listActiveAppliedLearningRules` ; reached via the facade.
- **Includes:** `BOOST_RANK` , `SCORE_BOOST` , `SCORE_PENALTY` only; affects **which jobs are planned**, not prompt text (`ranking_rules.ts`).

Other

- **Anti-repetition (carousel):** `buildLlmApprovalAntiRepetitionBlock` — `LLM_APPROVAL_ANTI_REPETITION_*` .
- **Rework:** reviewer notes + `editorial_overrides_json` in user prompt when `isEditorialRework` .
- **Attribution:** `caf_core.learning_generation_attribution` , `job_outcomes` (publish → performance anchor), run context snapshots via `setRunContextSnapshot()` .

11. Flow typing & special flow types

- `flow-kind.ts` : `isCarouselFlow` , `isVideoFlow` (regex + product video).
- `product-flow-types.ts` : `FLOW_PRODUCT_*` video; `FLOW_IMG_*` image flows **not** wired to generation.
- `offline-flow-types.ts` : excluded from planning/pipeline (e.g. some reel/hook variation names).
- `top-performer-mimic-flow-types.ts` :
 `FLOW_TOP_PERFORMER_MIMIC_IMAGE` ,
 `FLOW_TOP_PERFORMER_MIMIC_CAROUSEL` ,
 `FLOW_VISUAL_FIRST_CAROUSEL` , `FLOW_WHY_MIMIC_CAROUSEL` (render via `isTpGroundedCarouselRenderFlow()` when

`MIMIC_IMAGE_ENABLED=1`); `FLOW_TOP_PERFORMER_MIMIC_VIDEO` routes to HeyGen only.

- `new-visual-carousel-*.ts` : Visual-first lane — `execution_mode`: `new_visual` , idea + BVS, no TP `reference_items` .
- `brand-bible.ts` / `bvs-v1.ts` : Brand Visual System — versioned `brand_bibles` → `generation_payload.bvs_v1` .
- `format-routing.ts` : separate planning lanes `mimic_image` / `mimic_carousel` / `visual_first_carousel` parallel to standard carousel.

Top-performer mimic & carousel lanes (summary)

Recreate or reinterpret carousel/image patterns with fresh LLM copy. Upstream: Creative Intelligence ingest → `visual_guidelines_pack_v1` on signal packs → ideas grounded to `insights_id` (classic mimic) or idea + BVS (new visual).

Lane	Flow	execution_mode	Reference frames
Manual mimic carousel	<code>FLOW_TOP_PERFORMER_MIMIC_CAROUSEL</code>	<code>classic</code>	TP archived frames
New visual carousel	<code>FLOW_VISUAL_FIRST_CAROUSEL</code>	<code>new_visual</code>	None — idea + BVS T2I
Why Mimic carousel	<code>FLOW_WHY_MIMIC_CAROUSEL</code>	<code>why_mimic</code>	TP frames + SIL
Mimic image	<code>FLOW_TOP_PERFORMER_MIMIC_IMAGE</code>	—	Single frame

Phase	Module	Output on job
Pre-copy	<code>mimic-draft-prep.ts</code> Or <code>new-visual-carousel-prep.ts</code>	<code>mimic_v1</code> , <code>bvs_v1</code> , <code>mimic_render_context</code> , <code>mimic_job_grounding</code>
Copy	<code>llm-generator.ts</code> + mimic prompts	<code>generated_output</code> ; TP-grounded carousel → <code>mimic_carousel_package</code> snapshot
Render	<code>mimic-carousel-render.ts</code> , <code>mimic-image-job.ts</code> , <code>bvs-render-overlays.ts</code>	<code>MIMIC_BACKGROUND</code> , <code>MIMIC_VISUAL_PLATE</code> , <code>STATIC_IMAGE</code> assets

Modes (`mimic_v1.mode`): `image_full` , `template_bg` , `carousel_visual` .

Classifier: `mimic-mode-classifier.ts` (bypassed for `new_visual`).

Text invariant: TP-grounded carousels do **not** bake LLM copy into image models — overlay-only via DocAI/HBS + `reprint-text-overlay` (`job-pipeline.ts`).

Providers: `MIMIC_IMAGE_PROVIDER` default `bf1` ; copy always uses OpenAI. Full guide: `docs/MIMIC_FLOWS_COMPLETE_GUIDE.md` ; repo truth: `docs/CAF_CURRENT_STATE_CONTEXT_PACK.md` §11.

12. HTTP API surface (index)

Area	File
/v1/decisions, jobs, review queue, reviews	src/routes/v1.ts
/v1/runs/...	src/routes/runs.ts
/v1/signal-packs/...	src/routes/signal-packs.ts
/v1/pipeline/...	src/routes/pipeline.ts
/v1/projects/...	src/routes/project-config.ts
/v1/flow-engine/...	src/routes/flow-engine.ts
/v1/learning/...	src/routes/learning.ts
/v1/creative-intelligence/...	src/routes/creative-intelligence.ts
/v1/inputs-evidence/..., /v1/inputs-processing/...	inputs-evidence.ts, inputs-processing.ts
/v1/publications/...	src/routes/publications.ts
/admin, /v1/admin/...	src/routes/admin.ts
/api/templates/...	src/routes/renderer-templates.ts

Examples: `docs/API_REFERENCE.md` . Quick start: `README.md` .

13. Persistence (`src/repositories`)

Module	Role
core.ts	Projects, constraints, planning learning rules, prompt versions
jobs.ts	upsertContentJob, run-scoped deletes
runs.ts	Run CRUD, status
signal-packs.ts	Packs
flow-engine.ts	Flow defs, prompts, schemas, QC checks, listRiskPolicies
learning.ts, learning-evidence.ts	Rules + attribution
assets.ts, publications.ts, review-queue.ts, transitions.ts	As named

project-config.ts	Strategy, brand, platform, allowed flows, HeyGen, project risk rules (caf_core.risk_rules)
ops.ts	Audits, reviews, metrics inserts
db/queries.ts	q, qOne

14. Repository layout & companion services

Path	Role
src/server.ts	API entry
src/config.ts	Env schema
migrations/	SQL
apps/review/	Next.js workbench
services/renderer/	Carousel PNGs
services/video-assembly/	ffmpeg services
services/media-gateway/	Combined gateway

CLIs: `package.json` — `process-run`, `start-run:xlsx`, `migrate`, etc.

14.1 Observability

- **Structured pipeline log helper:** `src/services/pipeline-logger.ts` emits a single JSON line per event to stderr with `ts`, `level`, `stage`, `message`, and correlation fields (`project_id`, `run_id`, `task_id`, `job_id`, `flow_type`, `data`). Zero deps. Opt-in — existing `console.*` call sites still work; prefer the helper in new or touched code so a single job is easy to trace across generate → QC → render → review.
- **Swappable sink:** `setPipelineLogSink(fn)` is provided for tests (so asserting log output does not litter the Vitest stream).

15. Engineering invariants

1. `task_id` + `project_id` are the primary keys for joining jobs to drafts, assets, reviews, transitions.
2. Do not **double-submit HeyGen** when `render_state` already has `video_id` / `session_id`. The canonical check is

- `hasActiveProviderSession(renderState)` in `src/domain/content-job-render-state.ts` ; use it instead of re-implementing the check.
3. **generation_payload** changes affect pipeline, Review, admin — coordinate schema drift. Use `mergeGenerationPayloadQc` (`src/domain/generation-payload-qc.ts`) to write QC results; it is the only sanctioned writer for that slice.
 4. **Project risk rules** (`caf_core.risk_rules`) are not enforced by QC today; enforcement is **risk_policies** (now scoped by `applies_to_flow_type`) + **brand bans**. `GET /v1/projects/:slug/risk-qc-status` is the canonical "what does QC actually run here?" endpoint and must stay consistent with `qc-runtime.ts` .
 5. **Learning lookups** go through `src/services/learning-rule-selection.ts` . Calling `listActiveAppliedLearningRules` or `compileLearningContexts` directly from a new feature is considered drift — extend the facade instead.
 6. **Review app** `/v1/ contract` is covered by `src/routes/review-contract.test.ts` . Renaming or removing a listed path requires updating that test deliberately.
-

16. Split documentation map

For smaller files to edit in isolation:

Topic	File
Product pitch	<code>docs/CAF_PRODUCT_PITCH.md</code>
Complete product guide	<code>docs/CAF_COMPLETE_PRODUCT_GUIDE.md</code>
Overview	<code>docs/PROJECT_OVERVIEW.md</code>
Architecture (short)	<code>docs/ARCHITECTURE.md</code>
Lifecycle only	<code>docs/LIFECYCLE.md</code>
Stack only	<code>docs/TECH_STACK.md</code>
QC only	<code>docs/QUALITY_CHECKS.md</code>
Guidance only	<code>docs/GENERATION_GUIDANCE.md</code>
Risk only	<code>docs/RISK_RULES.md</code>

Per-layer pages	docs/layers/*.md
HTTP examples	docs/API_REFERENCE.md
Video / HeyGen details	docs/VIDEO_FLOWS.md, docs/HEYGEN_API_V3.md
Top-performer mimic	docs/MIMIC_FLOWS_COMPLETE_GUIDE.md, docs/MIMIC_IMAGE_FLOWS.md, docs/CREATIVE_INTELLIGENCE.md
AI agent onboarding	AGENTS.md
Domain IDs	docs/DOMAIN_MODEL.md (external) or .cursor/rules/caf-domain-model.mdc
External / ChatGPT bundle	docs/EXTERNAL_CONTEXT_PACK.md
Rebuild from docs	docs/REBUILD_FROM_DOCS.md
Database tables	docs/DATABASE_SCHEMA.md

17. External context pack (ChatGPT / other repos)

To give **another LLM or repository** enough context without uploading all source code:

1. Start with **docs/CAF_CURRENT_STATE_CONTEXT_PACK.md** (repo-derived July 2026 truth) and **docs/EXTERNAL_CONTEXT_PACK.md** — tiered file list + system prompt template.
2. **Tier 1 (always):** **AGENTS.md**, current-state pack (or volumes 11–14 PDFs), **docs/DOMAIN_MODEL.md**, **docs/ARCHITECTURE.md**, **docs/LIFECYCLE.md**, **docs/TECH_STACK.md**.
3. **Tier 2 (by topic):** mimic, inputs, video, API reference, QC, risk — see **EXTERNAL_CONTEXT_PACK** table.
4. **Tier 3 (implementation):** attach only the 3–15 relevant **src/** files for the task.

Do not paste **node_modules**, full **admin.ts**, or entire **migrations/** into ChatGPT.

For schema truth, use **docs/DATABASE_SCHEMA.md** + specific migration files.

18. Database schema summary

All tables live in PostgreSQL schema **caf_core**. 79 migrations through **078_brand_bibles.sql** (see **migrations/**).

Domain	Key tables
Pipeline	projects, runs, signal_packs, content_jobs, job_drafts, assets

Review/QC	editorial_reviews, diagnostic_audits, qc_checklists, risk_policies
Config	brand_constraints, brand_bibles, brand_profiles, allowed_flow_types, flow_definitions, prompt_templates
Learning	learning_rules, learning_observations, performance_metrics, job_outcomes
Publishing	publication_placements
Inputs	inputs_evidence_*, inputs_ideas, ideas, signal_pack_ideas
Creative Intel	creative_source_assets, creative_visual_analyses, creative_insights

Full catalog: [docs/DATABASE_SCHEMA.md](#) .

19. Rebuild from documentation

Another team can bootstrap a working stack using:

1. Postgres + `npm run migrate`
2. `.env` from `.env.example` (minimum: `DATABASE_URL` , `OPENAI_API_KEY`)
3. Optional: media workers + Review app
4. Seed flows + demo project
5. Ingest signal pack → materialize jobs → start → process → render → review

Step-by-step: [docs/REBUILD_FROM_DOCS.md](#) .

20. Inputs pipeline (upstream)

Research intake before signal packs:

Stage	Where	Tables
Evidence upload	Admin /admin/inputs, POST /v1/inputs-evidence/upload	inputs_evidence_imports, inputs_evidence_rows
Scrapers	Admin scrapers tab, npm run run-scraper	inputs_scraper_*
Insights	Admin /admin/processing	inputs_evidence_row_insights
Build signal pack	Processing API	signal_packs, ideas

Creative Intelligence	Ingest APIs	creative_*
-----------------------	-------------	------------

Review app: **pipeline** pages proxy read-only evidence/signal-pack inspect — processing controls stay in Core admin.

Roadmap: **docs/CAF_INPUTS_PIPELINE_ROADMAP.md** .

*Generated as a merged reference; when in doubt, trust the codebase and **migrations/** over this document.*